Model Selection

Cross-validation and the bias-variance trade-off

The problem

- One train/test split is noisy
- Tuning on the test set leaks information
- Complex models fit noise; simple ones miss signal

k-fold cross-validation

- Split data into k folds
- Train on k–1 folds, validate on the held-out one
- Repeat k times; report mean ± std
- Fresh, unfitted model every fold

cross_val_score

```
def cross_val_score(make_model, X, y, k=5, score=None, seed=0):
    """Fit a fresh model on each of k folds; return the k validation scores.
    make_model() builds an unfitted model; score(y_true, y_pred) defaults to accuracy."""
    X, y = np.asarray(X), np.asarray(y)
    if not 2 <= k <= len(X):
        raise ValueError("k must be between 2 and n_samples")
    score = score or (lambda t, p: float(np.mean(t == p)))
    out = []
    for tr, va in k_fold_indices(len(X), k, seed):
        m = make_model().fit(X[tr], y[tr])
        out.append(score(y[va], m.predict(X[va])))
    return np.array(out)
```

Bias-variance decomposition

$$E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$$

- Bias: error of the average model
- Variance: how much the model moves between datasets
- σ²: irreducible noise

Measuring it empirically

```
def bias_variance(make_model, f, degree, n_train=30, n_sets=200, noise=0.3, seed=0):
    """Train on n_sets noisy samples of f, evaluate on a fixed grid.
    Returns (bias^2, variance, noise^2) averaged over the grid."""
    rng = np.random.default_rng(seed)
    grid = np.linspace(0, 1, 50)
    Xg = polynomial_features(grid, degree)
    preds = np.empty((n_sets, grid.size))
    for s in range(n_sets):
        x = rng.uniform(0, 1, n_train)
        y = f(x) + rng.normal(0, noise, n_train)
        preds[s] = make_model().fit(polynomial_features(x, degree), y).predict(Xg)
    mean = preds.mean(0)
    return float(np.mean((mean - f(grid)) ** 2)), float(np.mean(preds.var(0))), noise ** 2
```

What you see

- Low degree: high bias, low variance (underfit)
- High degree: low bias, high variance (overfit)
- Sweet spot minimises their sum
- More data or regularisation lowers variance

Recap

- Use k-fold CV to compare and tune models
- Keep a final test set untouched
- Error = bias^2 + variance + noise
- Cost: k × training time